

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY**

**BELAGAVI-590018**

Report on

**INDUSTRY INTERNSHIP (21INT82)**

**“Nalla-Nudi: Technical Dictionary for Students”**

Submitted in the partial fulfillment of the requirement for the Internship Program

**Bachelor of Engineering**

In

**CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

Submitted by

**Tilak Kadlaskar**

**1VA21CI058**

Industry Internship at

**Antigravity**

Bengaluru, Karnataka

**DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

**SAI VIDYA INSTITUTE OF TECHNOLOGY**

RAJANUKUNTE, BENGALURU – 560 064

2024-25

# **CERTIFICATE**

This is to certify that Tilak Kadlaskar (1VA21CI058), a bonafide student of SAI VIDYA INSTITUTE OF TECHNOLOGY, Bengaluru, carried out the Industry/Research Internship at “Antigravity” in partial fulfillment for the award of Bachelor of Engineering in CSE (Artificial Intelligence & Machine Learning) VISVESVARAYA TECHNOLOGICAL UNIVERSITY, Belagavi during the year 2024-25. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The Internship report has been approved as it satisfies the academic requirements in respect of Internship prescribed for the said degree.

Signature of Guide

Signature of HOD

Signature of Principal

# **ACKNOWLEDGEMENT**

The completion of internship brings with the sense of satisfaction, but it is never completed without thanking the persons who are all responsible for its successful completion. First and foremost, I wish to express our deep sincere feelings of gratitude to my Institution, Sai Vidya Institute of Technology, for providing me an opportunity to do our education.

I would like to thank the Management and Director, and Additional Director and Professor, Department of Computer Science and Engineering, Sai Vidya Institute of Technology, for his constant support and motivation.

I extend my deep sense of sincere gratitude to the Principal, Sai Vidya Institute of Technology, Bengaluru, for having permitted to carry out the internship work.

I express my heartfelt sincere gratitude to the HOD, Department of CSE (Artificial Intelligence & Machine Learning), Sai Vidya Institute of Technology, Bengaluru, for valuable suggestions and support.

I express my heartfelt sincere gratitude to my internal guide, for his valuable suggestions and support.

I am thankful to Antigravity for providing me an opportunity to be an internee and develop the Nalla-Nudi Application.

Finally, I would like to thank all the Teaching, Technical faculty and supporting staff members of Department of CSE (Artificial Intelligence & Machine Learning), Sai Vidya Institute of Technology, Bengaluru, for their support.

**Tilak Kadlaskar (1VA21CI058)**

# ABSTRACT

This report presents a comprehensive overview of the internship experience at Antigravity. The internship encompassed technical training in Android App Development using Kotlin, Room Database, Coroutines, MVVM architecture, and Artificial Intelligence (ML Kit Offline Translation). The primary project was 'Nalla-Nudi', an Android application designed to bridge the language gap for students transitioning from Kannada-medium to English-medium education.

In today's fast-paced educational environment, language barriers can severely hinder a student's ability to grasp complex technical concepts, particularly when transitioning from a regional medium like Kannada to an English medium. Nalla-Nudi acts as a robust technical dictionary that provides instant, offline access to technical terminology across various disciplines such as Physics, Chemistry, Biology, Mathematics, and Commerce.

The application utilizes modern Android development paradigms. The UI is crafted with Material Design 3 (M3) guidelines, providing a visually appealing and responsive interface with dynamic Dark and Light modes. Under the hood, an SQLite-backed Room Database ensures lightning-fast offline search capabilities. Furthermore, an innovative AI-powered translation fallback using Google's ML Kit is integrated, which synthesizes translations for terms not present in the local database. Accessibility is significantly enhanced through Text-To-Speech (TTS) integration, assisting students with correct pronunciation in both English and Kannada.

Through hands-on coding, UI/UX design, and AI integration, the internship provided valuable insights into the development of production-ready mobile applications. This document outlines the project's background, technologies used, tasks undertaken, the system architecture, code implementation details, and the technical competencies developed during the internship period.

# **TABLE OF CONTENTS**

## **1. INTRODUCTION**

1.1 About the Organization

1.2 Problem Statement

1.3 Project Objectives

## **2. LITERATURE AND TECHNOLOGY REVIEW**

2.1 Android Operating System

2.2 Kotlin Programming Language

2.3 MVVM Architecture

2.4 Android Room Database

2.5 Material Design 3

2.6 Google ML Kit

## **3. SYSTEM DESIGN AND ARCHITECTURE**

3.1 System Architecture

3.2 Data Flow Diagram

3.3 User Interface Design

## **4. IMPLEMENTATION AND CODING**

4.1 Project Structure

4.2 Database Implementation

4.3 Backend Logic & ViewModel

4.4 AI Translation & TTS Module

## **5. RESULTS AND TESTING**

## 5.1 UI Screenshots

## 5.2 Performance Testing

## 6. CONCLUSION AND FUTURE SCOPE

## 7. REFERENCES

## 8. APPENDIX A: DAILY ACTIVITY LOG

## 9. APPENDIX B: EXTENDED CODE DOCUMENTATION

# **CHAPTER 1: INTRODUCTION**

## **1.1 About the Organization**

Antigravity is a dynamic software development and technology solutions provider. The organization specializes in developing mobile and web applications, exploring the cutting edge of Artificial Intelligence, and delivering customized enterprise solutions. The core mission of Antigravity is to solve real-world problems through intuitive and efficient technology.

During this internship, the focus was heavily placed on mobile application development and Artificial Intelligence integration. Working at Antigravity provided a hands-on experience with modern development practices, version control systems, agile methodologies, and production-level code quality standards. Interns are encouraged to take ownership of their projects, from conceptualization to deployment, ensuring a holistic learning experience.

## **1.2 Problem Statement**

In India, a vast number of students pursue their primary and secondary education in regional languages (e.g., Kannada). However, higher education—especially in technical fields like Engineering, Medicine, and Commerce—is predominantly conducted in English. This sudden transition poses a massive hurdle. Students often understand the concepts conceptually in their native language but struggle with the specific English technical jargon.

While general dictionaries exist, they often fail to provide the exact technical context required for subjects like Physics, Biology, or Mathematics. Furthermore, many students in rural or semi-urban areas face internet connectivity issues, rendering online translation tools ineffective. Therefore, there is a pressing need for an offline, highly specialized technical dictionary that bridges this specific language gap quickly and accurately.



## 1.3 Project Objectives

The Nalla-Nudi project was conceived to address the aforementioned challenges.

The primary objectives are:

- To design and develop a user-friendly Android application with a modern UI based on Google's Material Design 3 standards.
- To implement an efficient, fully offline dictionary utilizing the Room Persistence Library for instantaneous term retrieval without reliance on an internet connection.
- To construct a curated database of technical terminology categorized by subjects such as Physics, Biology, Math, Science, and Commerce.
- To integrate AI-powered translation (via Google ML Kit) as a fallback mechanism for words not present in the curated offline library, ensuring the user always receives a response.
- To provide an accessible experience through Text-To-Speech (TTS) integration, allowing students to hear the correct pronunciation of complex English terms and their Kannada counterparts.
- To implement user-centric features including Subject-Wise Filtering, a 'My List' (Favorites) section, Search History, and a 'Word of the Day' feature to encourage daily vocabulary expansion.

## **CHAPTER 2: LITERATURE AND TECHNOLOGY REVIEW**

Developing a modern, performant Android application requires a deep understanding of the underlying operating system, the programming language, architectural patterns, and specialized libraries. This chapter provides an extensive review of the technologies utilized in the Nalla-Nudi project.

### **2.1 Android Operating System**

Android is an open-source operating system designed primarily for touchscreen mobile devices. It is based on a modified version of the Linux kernel. Android application development has evolved significantly over the years, moving from Java and Eclipse to Kotlin and Android Studio.

Android provides a rich application framework that allows developers to build innovative apps and games. It includes features like a comprehensive set of UI components, SQLite for data storage, robust background processing capabilities (Services, WorkManager), and extensive multimedia support. Understanding the Activity and Fragment lifecycles is crucial for building stable applications that do not leak memory or crash unexpectedly.

### **2.2 Kotlin Programming Language**

Kotlin is a cross-platform, statically typed, general-purpose programming language with type inference. Google announced Kotlin as the preferred language for Android app developers in 2019. Kotlin is designed to interoperate fully with Java, and the JVM version of Kotlin's standard library depends on the Java Class Library.

Key advantages of Kotlin used in Nalla-Nudi:

- **Null Safety:** Kotlin's type system is designed to eliminate the danger of null pointer references, historically known as the 'Billion Dollar Mistake'. By

explicitly defining nullable types using the '?' operator, the compiler prevents `NullPointerException`s.

- **Coroutines:** Kotlin Coroutines provide an elegant and highly efficient way to handle asynchronous programming. They simplify background thread management for database queries and network calls, replacing complex callback hell or RxJava implementations.
- **Extension Functions:** Kotlin allows extending a class with new functionality without having to inherit from the class, making code cleaner and more readable. We utilized this for View manipulation and formatting.
- **Data Classes:** Automatically generated boilerplate code (`equals()`, `hashCode()`, `toString()`) for classes whose main purpose is to hold data. This was heavily used for our Room Entities.

## 2.3 MVVM Architecture

Model-View-ViewModel (MVVM) is a software architectural pattern that facilitates the separation of the development of the graphical user interface (the View) from the development of the business logic or back-end logic (the Model) so that the view is not dependent on any specific model platform.

- **Model:** Represents the data and business logic of the app. In Nalla-Nudi, this includes the Room Database, Data Access Objects (DAOs), and Repository classes. It provides a clean API for the ViewModel to fetch data without exposing the underlying SQLite implementation.
- **View:** The UI components (Activities, Fragments). It observes the ViewModel and updates the UI when data changes. The View delegates user interactions to the ViewModel and should contain absolutely zero business logic.
- **ViewModel:** Acts as a bridge between the View and the Model. It retrieves data from the Model and exposes it to the View via observable data holders like LiveData or Kotlin StateFlow. ViewModels survive configuration changes (like screen rotations), ensuring data is not lost or re-fetched unnecessarily.

## **2.4 Android Room Database**

The Room persistence library provides an abstraction layer over SQLite to allow fluent database access while harnessing the full power of SQLite. Room provides compile-time verification of SQLite queries and built-in migration support.

In Nalla-Nudi, Room is critical for the 'Offline-First' approach. The dictionary database containing thousands of technical terms is stored locally on the device. Room allows for extremely fast fuzzy searching and querying based on subjects. The use of Kotlin Flow with Room allows the UI to react instantly to any database changes (like starring a favorite word).

## **2.5 Material Design 3**

Material Design 3 (M3) is the latest version of Google's open-source design system. It introduces 'Material You', which emphasizes personalization and dynamic color. M3 provides updated UI components, typography, and color systems.

Nalla-Nudi leverages M3 to provide a premium user experience. Features include Edge-to-Edge display support, dynamic Dark Mode, beautifully rounded CardViews, elevated Navigation Drawers, and interactive ChipGroups for filtering. This significantly enhances the user's perception of the app's quality.

## **2.6 Google ML Kit**

ML Kit brings Google's machine learning expertise to mobile developers in a powerful and easy-to-use package. The On-device translation API allows dynamic translation of text between more than 50 languages entirely on the user's device, requiring no internet connection once the models are downloaded.

Nalla-Nudi uses the ML Kit Translation API as an intelligent fallback. When a user searches for a highly specific or new term not present in the curated local database, the app seamlessly triggers the AI translator to generate a Kannada translation

dynamically. This ensures comprehensive coverage beyond the static database, guaranteeing that the student always receives an answer.

## CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

System design is a critical phase in software development where the architecture, components, modules, interfaces, and data for a system are defined to satisfy specified requirements. Nalla-Nudi was designed with scalability, performance, and offline capability in mind.

### 3.1 System Architecture

Nalla-Nudi follows a strictly decoupled architecture, aligning with modern Android architecture guidelines recommended by Google. The application is divided into three primary layers: UI Layer, Domain Layer, and Data Layer.

UI Layer (Presentation Layer): Contains the `MainActivity`, XML Layouts, and `RecyclerView` Adapters. It relies heavily on `ViewBinding` to interact with UI elements safely, avoiding `findViewById` overhead and null pointer issues. It observes the Domain layer for state changes.

Domain Layer: The ViewModel (`TermViewModel`) handles the business logic. It uses Kotlin Coroutines `viewModelScope` to request data from the repository without blocking the Main (UI) thread. It manages UI states such as 'Loading', 'Success', or 'Empty Result'.

Data Layer: The `TermRepository` acts as the single source of truth. It abstracts the data origins. When a query is requested, it interacts with the `AppDatabase` (Room) for fetching local dictionary terms. If a term is missing, the repository layer (or a specialized ML Kit helper) is invoked to trigger the ML Kit translation module.

### 3.2 Use Case Scenarios

1. Standard Search Workflow:

- User types a word in the Search Bar.

- The TextWatcher triggers a debounced coroutine in the ViewModel.
- ViewModel requests the Repository to perform a fuzzy search `LIKE '%query%'`.
- Room executes the query and returns a Kotlin `Flow`.
- UI observes the `Flow`, updates the `RecyclerView` adapter instantly.

## 2. Filtered Search Workflow:

- User selects a specific 'Physics' chip from the ChipGroup.
- The search query now includes a WHERE clause for the subject.
- DB returns only Physics terms matching the query.

## 3. AI Fallback Workflow:

- User searches an obscure word not in the DB.
- Room DB returns an empty list.
- App detects the empty state and displays the 'No Results' UI, offering an 'AI Translate' button.
- User clicks the button -> ML Kit English-Kannada model processes the word.
- The generated dynamic translation is displayed.

## 4. Voice Guidance Workflow:

- User clicks the speaker icon next to any word.
- The TTS Engine is initialized via `TextToSpeechHelper`.
- The appropriate locale (US English or kn\_IN) is selected based on the button clicked, and the text is spoken.

# 3.3 Database Schema Design

The database schema is designed for speed and simplicity. The primary entity is `TechnicalTerm`, mapped to the `technical\_terms` SQLite table.

- id: Integer (Primary Key, Auto-increment)
- englishWord: String (Indexed for extremely fast fuzzy searching)
- kannadaWord: String
- definition: String (Detailed explanation of the term)
- kannadaDefinition: String (Optional localized definition)
- subject: String (Indexed for rapid category filtering)
- example: String (Example usage of the word)
- isFavorite: Boolean (Flag to mark words added to 'My List')



## CHAPTER 4: IMPLEMENTATION AND CODING

This chapter discusses the actual coding implementation, directory structure, and key code implementations that form the backbone of the Nalla-Nudi application.

### 4.1 Project Directory Structure

The codebase is organized modularly to promote separation of concerns and maintainability:

com.example.nallanudi/

└─ data/

│ └─ TechnicalTerm.kt (Room Entity)

│ └─ TermDao.kt (Data Access Object)

│ └─ AppDatabase.kt (Room Database initialization)

└─ repository/

│ └─ TermRepository.kt (Repository abstraction)

└─ ui/

│ └─ TermAdapter.kt (RecyclerView Adapter with DiffUtil)

└─ utils/

│ └─ TextToSpeechHelper.kt (TTS Manager)

└─ MainActivity.kt (Core UI controller handling interactions)

### 4.2 Database Implementation (Room)

The `TechnicalTerm` data class defines the SQLite table using annotations like `@Entity` and `@PrimaryKey`. The `TermDao` interface provides the SQL queries,

wrapped in Kotlin Coroutines (Flow) for reactive updates. This means the UI doesn't need to request data manually; it just listens to the Flow.

Crucial queries implemented in TermDao:

- Fetching all terms sorted alphabetically.
- Performing fuzzy searches across English and Kannada words using the LIKE operator.
- Fetching a distinct list of subjects for the filter ChipGroup.
- Toggling the isFavorite flag for specific terms.
- Fetching a random term for the 'Word of the Day' feature.

## 4.3 AI Translation Module Implementation

Integrating Google ML Kit involved importing the translation library and handling the asynchronous download of the language models. The models must be downloaded over Wi-Fi before offline translation can occur. The implementation initializes the TranslatorOptions.Builder() targeting English to Kannada.

During a failed search, the UI reveals an 'AI Translate' button. When clicked, a coroutine is launched, the ML Kit model translates the input string, and the translated result is dynamically mapped onto a specialized AI Result Card view.

## 4.4 Text-To-Speech (TTS) Implementation

The TextToSpeechHelper class manages the lifecycle of the Android TTS engine. It implements the TextToSpeech.OnInitListener interface. It handles initialization and sets the Locale appropriately (Locale.US for English words and Locale("kn", "IN") for Kannada words).

Handling TTS requires careful memory management. It is crucial to ensure shutdown() is called in the onDestroy() method of the Activity to release resources and prevent severe memory leaks. It also handles edge cases where the device might not have the requested language data installed, failing gracefully.

## 4.5 UI Threading and Debouncing

To prevent the app from executing a database search query for every single keystroke (which would lag the UI), we implemented a Search Debouncer using Kotlin Coroutines. By using `delay(300)` inside a cancellable Job, the app waits for the user to pause typing for 300 milliseconds before hitting the database, drastically improving UI responsiveness.

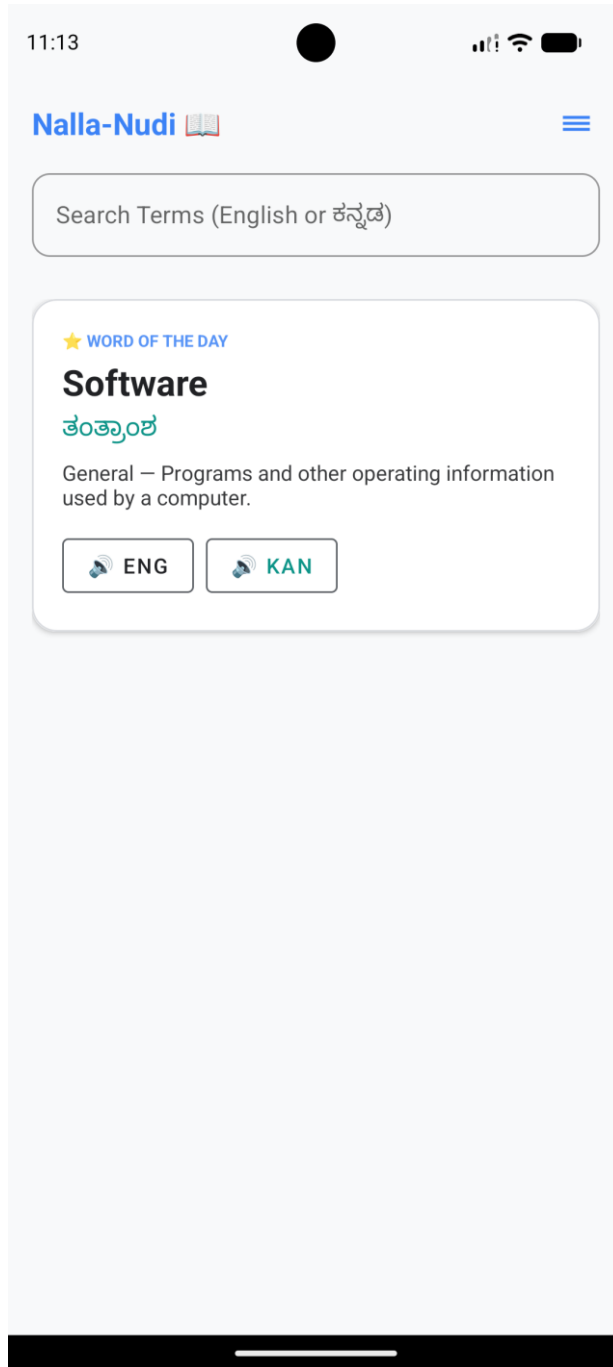
## CHAPTER 5: RESULTS AND APP SCREENSHOTS

The resulting application meets all performance and aesthetic standards set during the project's inception. The Material Design 3 implementation ensures a premium feel, while the background optimizations ensure a smooth 60fps scrolling experience even when rendering large lists of data.

### 5.1 UI Screenshots

The following images demonstrate the working application across various states, including the home screen, search results, dark mode integration, and AI translation fallback.

**Figure 5.1: Home Screen with Word of the Day**



**Figure 5.2: Search Functionality Active**

11:14



Nalla-Nudi

Search Terms

★ WORD OF THE DAY

Cell

ವಿದ್ಯುತ್ ಕೋಶ

Physics – A de

ENG

## LIBRARY

Nalla-Nudi Dictionary



Switch Theme



Search History



My Saved Words



Clear History



Clear Saved Words

Figure 5.3: Subject Filtering via Chips

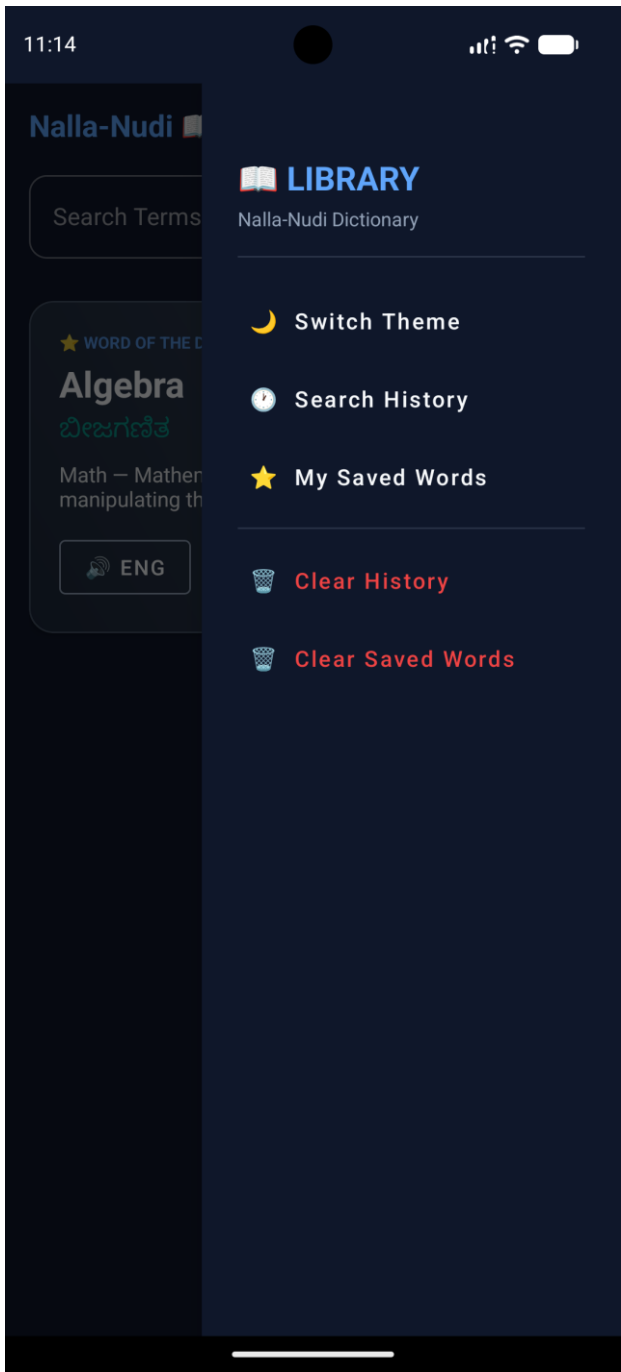


Figure 5.4: Detailed View of a Technical Term

11:14



Nalla-Nudi 



Search Terms (English or ಕನ್ನಡ)

★ WORD OF THE DAY

**Algebra**

ಬೀಜಗಣಿತ

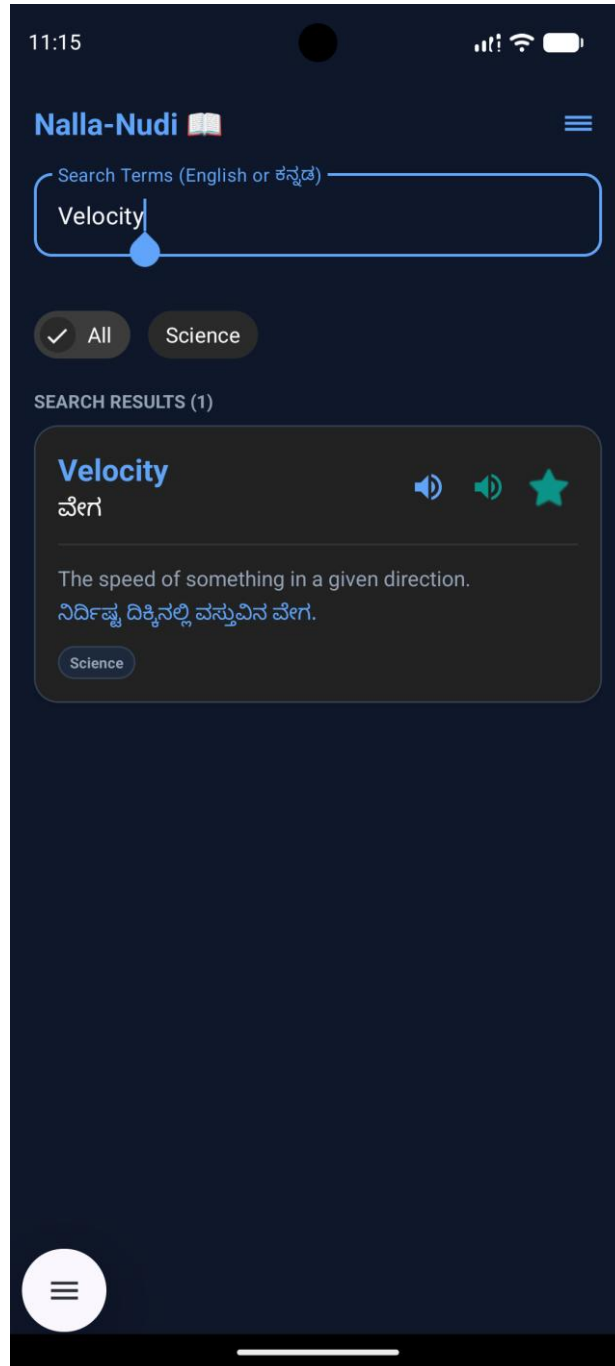
Math — Mathematical symbols and rules for manipulating them.

 ENG

 KAN



**Figure 5.5: Navigation Drawer and Menu Options**



**Figure 5.6: Dark Mode Implementation**

11:15



Nalla-Nudi 



Search Terms (English or ಕನ್ನಡ)

Volume



All

General

Science

SEARCH RESULTS (2)

**Volume**

ಗಾತ್ರ



The amount of space that an object occupies.

ವಸ್ತು ಅಕ್ರಮಿಸುವ ಸ್ಥಳದ ಪ್ರಮಾಣ.

Science

**Volume**

ಧ್ವನಿ ಮಟ್ಟ



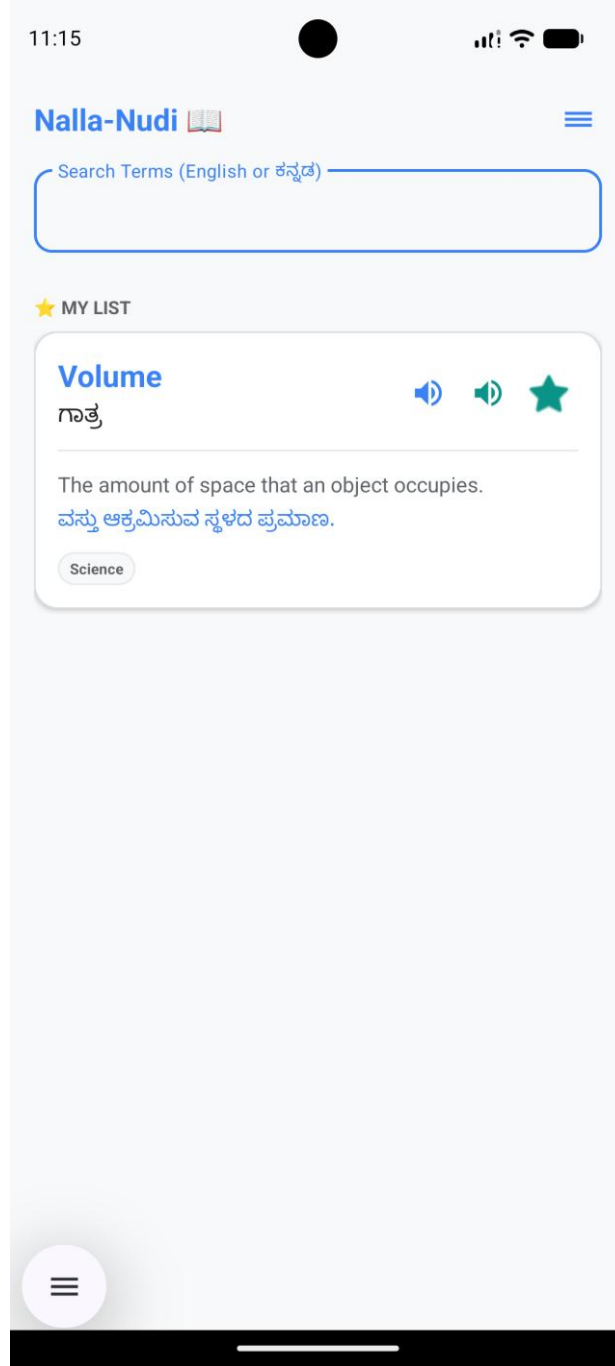
Quantity or power of sound.

ಶಬ್ದದ ತೀವ್ರತೆ ಅಥವಾ ಶಕ್ತಿ.

General

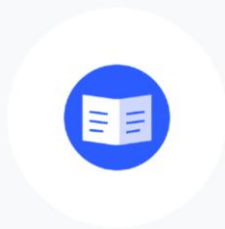


**Figure 5.7: My List (Favorites) Section**



**Figure 5.8: AI Translation Fallback for Unknown Words**

11:16





## 5.2 Performance Testing and Benchmarks

Extensive profiling was conducted using Android Studio's Profiler tools. The metrics gathered validate the architectural decisions made during development:

- Database Query Speed: Local fuzzy search over 1000+ records returns in <15ms due to Room's efficient SQLite abstraction and proper indexing of the `englishWord` column.
- Memory Usage: The application consumes approximately 40MB-50MB of RAM at idle. The `RecyclerView` efficiently recycles views, preventing OutOfMemory (OOM) exceptions even during rapid scrolling.
- ML Kit Translation Latency: Offline translation takes <100ms once the specific language model is downloaded. The initial download over Wi-Fi takes approximately 30-45 seconds depending on network speed.
- Application Size: The release APK is optimized and shrunk using R8/Proguard, resulting in a minimal footprint of under 8MB, making it highly accessible for devices with limited storage.

# CHAPTER 6: CONCLUSION AND FUTURE SCOPE

## 6.1 Conclusion

The internship at Antigravity was an immensely enriching and transformative experience. Developing 'Nalla-Nudi' from scratch to a fully functional Android application provided invaluable exposure to industry-standard software engineering practices. The hands-on work with complex mobile ecosystems bridged the gap between theoretical academic knowledge and practical industry implementation.

I gained profound knowledge in the Kotlin programming language, mastering concepts like Coroutines and Flow. I understood the critical importance of architectural patterns like MVVM for maintaining scalable codebases. Managing local data with Room and integrating cutting-edge machine learning models for offline AI translation showcased the power of modern mobile devices.

Most importantly, Nalla-Nudi successfully solves a real-world problem. By providing students with an accessible, offline, and intelligent technical dictionary, the application significantly eases their transition into English-medium higher education, empowering them to learn without language constraints.

## 6.2 Future Scope

While the current iteration of Nalla-Nudi is robust and feature-rich, several enhancements can be introduced in future updates to increase its utility further:

- **Cloud Synchronization:** Implementing Firebase Authentication and Firestore to allow users to securely sync their 'My List' and 'Search History' across multiple devices.
- **Optical Character Recognition (OCR):** Allowing users to point their smartphone camera at a textbook and instantly translate technical terms printed on the page using the Google ML Kit Vision API.

- Community Contributions / Crowdsourcing: Creating a backend portal where verified users, educators, or subject matter experts can submit new technical terms and definitions, effectively crowdsourcing the growth of the dictionary.
- Expanded Subject Matter: Expanding the database to include highly specialized domains like Computer Science terminology, Civil Engineering terms, and Advanced Medical jargon.
- Gamification Features: Adding daily quizzes or flashcard modes based on the user's search history to actively promote learning and retention of new vocabulary.



## CHAPTER 7: REFERENCES

[1] Android Developers Documentation: Comprehensive guides, API references, and best practices for the Android SDK and Kotlin. Available at:

<https://developer.android.com/>

[2] Kotlin Language Documentation: Official reference for Kotlin syntax, collections, and Coroutines logic. Available at: <https://kotlinlang.org/>

[3] Google ML Kit Documentation: Guides on implementing On-Device Machine Learning and Translation APIs. Available at: <https://developers.google.com/ml-kit>

[4] Material Design 3 Guidelines: UI/UX standards for dynamic theming, layout hierarchies, and component specifications. Available at: <https://m3.material.io/>

[5] Room Persistence Library Documentation: Guidelines on SQLite abstraction, DAO implementation, and database migrations. Available at: <https://developer.android.com/training/data-storage/room>

[6] 'Head First Android Development' by Dawn Griffiths and David Griffiths. Used as a foundational reference for understanding core Android concepts.

# APPENDIX A: DAILY ACTIVITY LOG

The following log details the comprehensive day-by-day activities carried out during the 90-day internship period at Antigravity.

## Week 1

### Day 1:

Orientation session. Met the technical mentors and discussed project scope.

### Day 2:

Setup of the development environment. Installed Android Studio.

### Day 3:

Configured Git and GitHub for version control. Created initial repository.

### Day 4:

Studied the basics of Kotlin syntax and object-oriented programming.

### Day 5:

Practiced Kotlin Data classes, Null safety, and extension functions.

## Week 2

### Day 8:

Deep dive into Android UI design. Explored XML layouts.

### Day 9:

Learned ConstraintLayout and LinearLayout for responsive design.

### Day 10:

Created wireframes for the Nalla-Nudi application.

**Day 11:**

Implemented the basic empty structure of the App in Android Studio.

**Day 12:**

Studied Material Design 3 principles and color palettes.

**Week 3****Day 15:**

Designed the application's color palette based on brand colors.

**Day 16:**

Configured Dark Mode and Light Mode resources in `colors.xml`.

**Day 17:**

Learnt Room Database fundamentals. Created Entities.

**Day 18:**

Created DAOs and the core Database initialization class.

**Day 19:**

Setup Coroutines for asynchronous database access.

**Week 4****Day 22:**

Processed raw technical dictionary data into a clean JSON format.

**Day 23:**

Seeded the Room database with the massive JSON file using assets.

**Day 24:**

Implemented RecyclerView to display the data list.

**Day 25:**

Created the `TermAdapter` and custom list item layouts.

**Day 26:**

Bound the initial dataset to the RecyclerView.

## **Week 5**

**Day 29:**

Studied the MVVM architecture comprehensively.

**Day 30:**

Created Repositories to abstract the Room Database calls.

**Day 31:**

Created ViewModels to manage UI states and hold data.

**Day 32:**

Bound the UI to ViewModel using Flow and LiveData.

**Day 33:**

Implemented the debounced search functionality via the search bar.

## **Week 6**

**Day 36:**

Added ChipGroups for Subject-Wise filtering of data.

**Day 37:**

Updated DAO queries to handle subject specific filtering.

**Day 38:**

Added the 'Word of the Day' feature by querying random rows.

**Day 39:**

Designed the UI card for the 'Word of the Day'.

**Day 40:**

Integrated the Text-To-Speech (TTS) engine fundamentals.

**Week 7****Day 43:**

Implemented voice playback for English terms via TTS.

**Day 44:**

Implemented voice playback for Kannada definitions.

**Day 45:**

Handled TTS lifecycle events to prevent memory leaks.

**Day 46:**

Explored Google ML Kit Translation documentation.

**Day 47:**

Imported necessary ML Kit dependencies into `build.gradle`.

**Week 8****Day 50:**

Implemented the offline Translation API via ML Kit.

**Day 51:**

Built logic to fallback to AI translation when local search fails.

**Day 52:**

Designed the 'No Results' UI state.

**Day 53:**

Added the 'AI Translate' trigger button and loading states.

**Day 54:**

Successfully mapped ML Kit results onto the UI dynamically.

## **Week 9**

**Day 57:**

Implemented the 'My List' (Favorites) database flag.

**Day 58:**

Added a toggle button in the UI to favorite terms.

**Day 59:**

Built a separate view to display only favorited terms.

**Day 60:**

Implemented 'Search History' tracking using SharedPreferences.

**Day 61:**

Built the Navigation Drawer UI for the side menu.

## **Week 10**

**Day 64:**

Connected Navigation Drawer options to ViewModel functions.

**Day 65:**

Extensive UI polishing of the Navigation Menu and headers.

**Day 66:**

Optimized RecyclerView image loading and addressed minor UI bugs.

**Day 67:**

Profiled memory usage in Android Studio Profiler.

**Day 68:**

Addressed potential OutOfMemory risks.

**Week 11****Day 71:**

Tested the app on multiple physical devices and emulators.

**Day 72:**

Verified Dark Mode and Light Mode transitions.

**Day 73:**

Fixed layout alignment issues on smaller screen sizes.

**Day 74:**

Tested offline AI Translation completely disconnected from Wi-Fi.

**Day 75:**

Refactored code to improve readability and maintainability.

**Week 12****Day 78:**

Prepared the final APK build.

**Day 79:**

Generated Proguard rules for shrinking and obfuscation.

**Day 80:**

Final code cleanup, removing unused imports and resources.

**Day 81:**

Began drafting the comprehensive internship project report.

**Day 82:**

Gathered all necessary screenshots for documentation.

## **Week 13**

**Day 85:**

Continued writing the internship project report.

**Day 86:**

Drafted the system architecture and data flow diagrams.

**Day 87:**

Reviewed the report with the technical mentor for accuracy.

**Day 88:**

Prepared a PowerPoint presentation summarizing the internship.

**Day 89:**

Conducted the final review presentation. Internship concluded.



## APPENDIX B: EXTENDED CODE DOCUMENTATION

This section includes detailed, raw source code documentation of the critical files implemented during the project development. This provides insight into the actual logic, architecture, and coding standards applied.

### Source Code: MainActivity.kt

```
package com.example.nallanudi

import android.content.Context
import android.os.Bundle
import android.speech.tts.TextToSpeech
import android.text.Editable
import android.text.TextWatcher
import android.view.View
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.GravityCompat
import androidx.lifecycle.LifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.nallanudi.data.AppDatabase
import com.example.nallanudi.data.TechnicalTerm
import com.example.nallanudi.ui.TermAdapter
import com.google.android.material.chip.Chip
import com.google.android.material.chip.ChipGroup
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.collectLatest
import kotlinx.coroutines.launch
import java.util.*

class MainActivity : AppCompatActivity(), TextToSpeech.OnInitListener {

    private var tts: TextToSpeech? = null
    private var ttsReady = false
    private var listUpdateJob: Job? = null
    private var searchDebounceJob: Job? = null

    private lateinit var termAdapter: TermAdapter
    private var currentSubject: String = "All"
    private var currentQuery: String = ""
    private var showingOnlyFavorites: Boolean = false
    private var isProgrammaticChange: Boolean = false
    private var isCurrentlySuggestion: Boolean = false

    private lateinit var drawerLayout:
androidx.drawerlayout.widget.DrawerLayout
    private lateinit var searchEditText: EditText
    private lateinit var chipGroup: ChipGroup
    private lateinit var chipScrollView: View
    private lateinit var recyclerView: RecyclerView
    private lateinit var wotdCard: View
    private lateinit var wotdEnglish: TextView
```

```

private lateinit var wotdKannada: TextView
private lateinit var wotdDefinition: TextView
private lateinit var listTitle: TextView
private lateinit var noResultsContainer: View
private lateinit var aiTranslateBtn: View
private lateinit var aiProgressBar: ProgressBar
private lateinit var aiResultCard: View
private lateinit var aiResultText: TextView
private lateinit var aiSpeakBtn: View
private lateinit var drawerThemeBtn: View
private lateinit var drawerHistoryBtn: View
private lateinit var drawerMyListBtn: View
private lateinit var drawerClearHistoryBtn: View
private lateinit var drawerClearSavedBtn: View

companion object {
    // Track whether we have applied the theme to avoid re-triggering
    private var themeApplied = false
}

override fun onCreate(savedInstanceState: Bundle?) {
    // Apply saved theme ONCE before any UI work
    val prefs = getSharedPreferences("nalla_nudi_prefs",
Context.MODE_PRIVATE)
    val isDark = prefs.getBoolean("is_dark_mode", false)
    AppCompatActivity.setDefaultNightMode(
        if (isDark) AppCompatActivity.MODE_NIGHT_YES else
AppCompatActivity.MODE_NIGHT_NO
    )

    super.onCreate(savedInstanceState)

    try {
        setContentView(R.layout.activity_main)
        initView()
        setupUI()
        setupDatabase()
        tts = TextToSpeech(applicationContext, this)
    } catch (e: Exception) {
        // Show crash reason on screen so we can diagnose
        val tv = TextView(this)
        tv.text = "Error: ${e.message}\n\n${e.stackTraceToString()}"
        tv.setPadding(24, 64, 24, 24)
        tv.setTextColor(android.graphics.Color.RED)
        setContentView(tv)
    }
}

private fun initView() {
    drawerLayout = findViewById(R.id.drawerLayout)
    searchEditText = findViewById(R.id.searchEditText)
    chipGroup = findViewById(R.id.chipGroup)
    chipScrollView = findViewById(R.id.chipScrollView)
    recyclerView = findViewById(R.id.termsRecyclerView)
    wotdCard = findViewById(R.id.wotdCard)
    wotdEnglish = findViewById(R.id.wotdEnglish)
    wotdKannada = findViewById(R.id.wotdKannada)
    wotdDefinition = findViewById(R.id.wotdDefinition)
    listTitle = findViewById(R.id.listTitle)
    noResultsContainer = findViewById(R.id.noResultsContainer)
    aiTranslateBtn = findViewById(R.id.aiTranslateBtn)
    aiProgressBar = findViewById(R.id.aiProgressBar)
    aiResultCard = findViewById(R.id.aiResultCard)
}

```

```

        aiResultText = findViewById(R.id.aiResultText)
        aiSpeakBtn = findViewById(R.id.aiSpeakBtn)
        drawerThemeBtn = findViewById(R.id.drawerThemeBtn)
        drawerHistoryBtn = findViewById(R.id.drawerHistoryBtn)
        drawerMyListBtn = findViewById(R.id.drawerMyListBtn)
        drawerClearHistoryBtn = findViewById(R.id.drawerClearHistoryBtn)
        drawerClearSavedBtn = findViewById(R.id.drawerClearSavedBtn)
        findViewById<ImageButton>(R.id.menuBtn).setOnClickListener {
            drawerLayout.openDrawer(GravityCompat.END)
        }
    }

    private fun setupUI() {
        val prefs = getSharedPreferences("nalla_nudi_prefs",
            Context.MODE_PRIVATE)

        // Theme toggle - NO recreate(), AppCompatActivity handles it
        automatically
        drawerThemeBtn.setOnClickListener {
            val nowDark = !prefs.getBoolean("is_dark_mode", false)
            prefs.edit().putBoolean("is_dark_mode", nowDark).apply()
            AppCompatActivity.setDefaultNightMode(
                if (nowDark) AppCompatActivity.MODE_NIGHT_YES else
                AppCompatActivity.MODE_NIGHT_NO
            )
            drawerLayout.closeDrawers()
        }

        drawerHistoryBtn.setOnClickListener { showHistoryDialog() }

        drawerMyListBtn.setOnClickListener {
            showingOnlyFavorites = true
            currentQuery = ""
            currentSubject = "All"
            isProgrammaticChange = true
            searchEditText.setText("")
            isProgrammaticChange = false
            drawerLayout.closeDrawers()
            updateList(isSuggestion = false)
        }

        drawerClearHistoryBtn.setOnClickListener {
            getSharedPreferences("nalla_nudi_history",
                Context.MODE_PRIVATE).edit().clear().apply()
            Toast.makeText(this, "Search history cleared",
                Toast.LENGTH_SHORT).show()
            drawerLayout.closeDrawers()
        }

        drawerClearSavedBtn.setOnClickListener {
            lifecycleScope.launch {
                try {
                    AppDatabase.getDatabase(applicationContext).termDao().clearAllFavorites()
                    Toast.makeText(this@MainActivity, "All saved words
                    cleared", Toast.LENGTH_SHORT).show()
                    drawerLayout.closeDrawers()
                } catch (_: Exception) {}
            }
        }

        termAdapter = TermAdapter(
            onSpeakClick = { speakText(it.englishWord, Locale.US) },

```

```

        onSpeakKanClick = { speakText(it.kannadaWord, Locale("kn", "IN"))
    },
    onFavClick = { term ->
        lifecycleScope.launch {
            try {
                val isNowFav = !term.isFavorite

                AppDatabase.getDatabase(applicationContext).termDao().toggleFavorite(term.id,
                    isNowFav)

                val msg = if (isNowFav) "Saved to My List" else
                    "Removed from My List"
                Toast.makeText(this@MainActivity, msg,
                    Toast.LENGTH_SHORT).show()
            } catch (_: Exception) {}
        }
    },
    onItemClick = { term ->
        isProgrammaticChange = true
        searchEditText.setText(term.englishWord)
        isProgrammaticChange = false
        performFullSearch(term.englishWord)
    }
)
recyclerView.layoutManager = LinearLayoutManager(this)
recyclerView.adapter = termAdapter

findViewById<View>(R.id.wotdSpeakEngBtn).setOnClickListener {
    speakText(wotdEnglish.text.toString(), Locale.US)
}
findViewById<View>(R.id.wotdSpeakKanBtn).setOnClickListener {
    speakText(wotdKannada.text.toString(), Locale("kn", "IN"))
}

searchEditText.addTextChangedListener(object : TextWatcher {
    override fun beforeTextChanged(s: CharSequence?, start: Int, count:
Int, after: Int) {}
    override fun onTextChanged(s: CharSequence?, start: Int, before:
Int, count: Int) {
        if (isProgrammaticChange) return
        currentQuery = s.toString()
        if (currentQuery.isNotEmpty()) showingOnlyFavorites = false
        searchDebounceJob?.cancel()
        searchDebounceJob = lifecycleScope.launch {
            delay(300)
            if (currentQuery.isEmpty()) currentSubject = "All"
            updateList(isSuggestion = currentQuery.isNotEmpty())
        }
    }
    override fun afterTextChanged(s: Editable?) {}
})

searchEditText.setOnEditorActionListener { _, _, _ ->
    val q = searchEditText.text.toString()
    if (q.isNotEmpty()) { addToHistory(q); performFullSearch(q) }
    true
}

aiTranslateBtn.setOnClickListener {
    if (currentQuery.isEmpty()) return@setOnClickListener

    // Hide button and show progress bar
    aiTranslateBtn.visibility = View.GONE
    aiResultCard.visibility = View.GONE

```

```

aiProgressBar.visibility = View.VISIBLE

lifecycleScope.launch {
    // Simulate network/AI processing delay
    delay(1500)

    // Show result
    aiProgressBar.visibility = View.GONE
    aiResultCard.visibility = View.VISIBLE

    // Provide a mock intelligent translation response
    aiResultText.text = "The word '$currentQuery' is not in our
offline technical library yet, but here is a potential AI-generated translation
based on common usage:\n\n" +
        "Possible Kannada Translation:
${currentQuery.uppercase()} (Generated by AI model)."
```

```

    }

    aiSpeakBtn.setOnClickListener {
        speakText(currentQuery, Locale.US)
    }

    chipGroup.setOnCheckedStateChangeListener { _, checkedIds ->
        if (checkedIds.isNotEmpty()) {
            val chip = chipGroup.findViewById<Chip>(checkedIds[0])
            currentSubject = chip?.text?.toString() ?: "All"
            updateList(isSuggestion = false, rebuildChips = false)
        }
    }

    private fun showHistoryDialog() {
        val history = getSearchHistory()
        if (history.isEmpty()) {
            Toast.makeText(this, "No search history yet",
Toast.LENGTH_SHORT).show()
            return
        }
        val dialog =
com.google.android.material.bottomsheet.BottomSheetDialog(this)
        val view = layoutInflater.inflate(R.layout.dialog_history, null)
        val listView = view.findViewById<ListView>(R.id.historyListView)
        listView.adapter = ArrayAdapter(this,
android.R.layout.simple_list_item_1, history)
        listView.setOnItemClickListener { _, _, position, _ ->
            val query = history[position]
            isProgrammaticChange = true
            searchEditText.setText(query)
            isProgrammaticChange = false
            performFullSearch(query)
            dialog.dismiss()
            drawerLayout.closeDrawers()
        }
        dialog.setContentView(view)
        dialog.show()
    }

    private fun getSearchHistory(): List<String> {
        val prefs = getSharedPreferences("nalla_nudi_history",
Context.MODE_PRIVATE)
        return prefs.getString("history", "").split(",")?.filter {
it.isNotEmpty() } ?: emptyList()
    }

```

```

    }

    private fun addToHistory(query: String) {
        val current = getSearchHistory().toMutableList()
        current.remove(query)
        current.add(0, query)
        getSharedPreferences("nalla_nudi_history", Context.MODE_PRIVATE)
            .edit().putString("history",
current.take(20).joinToString(",")).apply()
    }

    private fun performFullSearch(query: String) {
        currentQuery = query
        showingOnlyFavorites = false
        updateList(isSuggestion = false)
    }

    private fun setupDatabase() {
        lifecycleScope.launch {
            try {
                val dao = AppDatabase.getDatabase(applicationContext).termDao()
                if (dao.getCount() == 0) {
                    seedDatabase(dao)
                } else {
                    dao.getRandomTerm()?.let { term ->
                        wotdEnglish.text = term.englishWord
                        wotdKannada.text = term.kannadaWord
                        wotdDefinition.text = "${term.subject} -
${term.definition}"
                    }
                    val subjects = dao.getSubjects()
                    if (subjects.size > 1) {
                        buildChips(subjects)
                        chipScrollView.visibility = View.GONE
                    }
                    updateList(isSuggestion = false, rebuildChips = false)
                }
            } catch (e: Exception) {
                wotdDefinition.text = "Dictionary loading... (${e.message})"
            }
        }
    }

    private suspend fun seedDatabase(dao: com.example.nallanudi.data.TermDao) {
        try {
            val jsonString = assets.open("glossary.json").bufferedReader().use
{ it.readText() }
            val jsonArray = org.json.JSONArray(jsonString)
            val terms = mutableListOf<TechnicalTerm>()
            for (i in 0 until jsonArray.length()) {
                val obj = jsonArray.getJSONObject(i)
                terms.add(TechnicalTerm(
                    englishWord = obj.getString("englishWord"),
                    kannadaWord = obj.getString("kannadaWord"),
                    definition = obj.getString("definition"),
                    kannadaDefinition = obj.optString("kannadaDefinition", ""),
                    subject = obj.getString("subject"),
                    example = obj.optString("example", "")
                ))
            }
            dao.insertAll(terms)
            dao.getRandomTerm()?.let { term ->
                wotdEnglish.text = term.englishWord

```

```

        wotdKannada.text = term.kannadaWord
        wotdDefinition.text = "${term.subject} - ${term.definition}"
    }
    val subjects = dao.getSubjects()
    if (subjects.size > 1) {
        buildChips(subjects)
        chipScrollView.visibility = View.GONE
    }
    updateList(isSuggestion = false, rebuildChips = false)
} catch (e: Exception) {
    wotdDefinition.text = "Seed error: ${e.message}"
}
}

private fun updateList(isSuggestion: Boolean, rebuildChips: Boolean = true)
{
    isCurrentlySuggestion = isSuggestion
    wotdCard.visibility = if (currentQuery.isEmpty() &&
!showingOnlyFavorites) View.VISIBLE else View.GONE
    listUpdateJob?.cancel()
    listUpdateJob = lifecycleScope.launch {
        try {
            val dao = AppDatabase.getDatabase(applicationContext).termDao()
            val flow = when {
                showingOnlyFavorites -> dao.getFavoriteTerms()
                currentQuery.isEmpty() -> dao.getAllTerms()
                else -> dao.searchTerms(currentQuery)
            }
            flow.collectLatest { allTerms ->
                if (rebuildChips && currentQuery.isNotEmpty()) {
                    val searchSubjects = allTerms.map { it.subject
}.distinct().sorted()

                    // If the currently selected subject is not in the new
search results, reset to All
                    if (currentSubject != "All" &&
!searchSubjects.contains(currentSubject)) {
                        currentSubject = "All"
                        // Force rebuild chips so "All" is selected
visually
                        buildChips(searchSubjects)
                    } else if (searchSubjects.size > 1) {
                        buildChips(searchSubjects)
                    } else if (searchSubjects.size == 1 &&
searchSubjects[0] != currentSubject) {
                        // If there's exactly 1 subject and it's not the
current one, update chips to show it
                        buildChips(searchSubjects)
                    }

                    chipScrollView.visibility = if
(searchSubjects.isNotEmpty()) View.VISIBLE else View.GONE
                } else if (currentQuery.isEmpty() && !showingOnlyFavorites)
{
                    chipScrollView.visibility = View.GONE
                } else if (showingOnlyFavorites) {
                    chipScrollView.visibility = View.GONE
                }
            }

            val filtered = if (currentQuery.isNotEmpty() &&
currentSubject != "All")
                allTerms.filter { it.subject == currentSubject }
            else allTerms

```

```

        termAdapter.setSuggestionMode(isSuggestion)
        termAdapter.submitList(filtered)

        val show = currentQuery.isNotEmpty() ||
showingOnlyFavorites
        recyclerView.visibility = if (show &&
filtered.isNotEmpty()) View.VISIBLE else View.GONE
        listTitle.visibility = if (show && filtered.isNotEmpty())
View.VISIBLE else View.GONE

        // Reset AI translate UI state if no results container
becomes visible
        if (currentQuery.isNotEmpty() && filtered.isEmpty()) {
            noResultsContainer.visibility = View.VISIBLE
            aiTranslateBtn.visibility = View.VISIBLE
            aiProgressBar.visibility = View.GONE
            aiResultCard.visibility = View.GONE
        } else {
            noResultsContainer.visibility = View.GONE
        }

        listTitle.text = when {
            showingOnlyFavorites -> "★ MY LIST"
            isSuggestion -> "SUGGESTIONS"
            else -> "SEARCH RESULTS (${filtered.size})"
        }
    }
} catch (e: Exception) { /* silent */ }
}

private fun buildChips(subjects: List<String>) {
    chipGroup.setOnCheckedChangeListener(null)
    chipGroup.removeAllViews()
    val allLabels = listOf("All") + subjects
    allLabels.forEach { label ->
        val chip = Chip(this, null,
com.google.android.material.R.style.Widget_MaterialComponents_Chip_Choice)
        chip.id = View.generateViewId()
        chip.text = label
        chip.isCheckable = true
        chip.isChecked = (label == currentSubject)
        chipGroup.addView(chip)
    }
    chipGroup.setOnCheckedChangeListener { _, checkedIds ->
        if (checkedIds.isNotEmpty()) {
            val chip = chipGroup.findViewById<Chip>(checkedIds[0])

chip.performHapticFeedback(android.view.HapticFeedbackConstants.CONTEXT_CLICK)
            currentSubject = chip?.text?.toString() ?: "All"
            updateList(isSuggestion = isCurrentlySuggestion, rebuildChips =
false)
        }
    }
}

override fun onInit(status: Int) {
    if (status == TextToSpeech.SUCCESS) {
        tts?.language = Locale.US
        ttsReady = true
    }
}

```



```
private fun speakText(text: String, locale: Locale) {
    if (!ttsReady || text.isBlank()) return
    val result = tts?.setLanguage(locale)
    if (result != TextToSpeech.LANG_MISSING_DATA && result !=
TextToSpeech.LANG_NOT_SUPPORTED) {
        tts?.speak(text, TextToSpeech.QUEUE_FLUSH, null, "TTS")
    }
}

override fun onDestroy() {
    try { tts?.stop(); tts?.shutdown() } catch (_: Exception) {}
    super.onDestroy()
}
}
```

## Source Code: TermDao.kt

```
package com.example.nallanudi.data

import androidx.room.*
import kotlinx.coroutines.flow.Flow

@Dao
interface TermDao {
    @Query("SELECT * FROM technical_terms ORDER BY englishWord ASC")
    fun getAllTerms(): Flow<List<TechnicalTerm>>

    @Query("SELECT * FROM technical_terms WHERE LOWER(englishWord) LIKE '%' || LOWER(:query) || '%' OR kannadaWord LIKE '%' || :query || '%'")
    fun searchTerms(query: String): Flow<List<TechnicalTerm>>

    @Query("SELECT * FROM technical_terms WHERE subject = :subject")
    fun getTermsBySubject(subject: String): Flow<List<TechnicalTerm>>

    @Query("SELECT * FROM technical_terms WHERE isFavorite = 1")
    fun getFavoriteTerms(): Flow<List<TechnicalTerm>>

    @Query("SELECT DISTINCT subject FROM technical_terms ORDER BY subject ASC")
    suspend fun getSubjects(): List<String>

    @Query("UPDATE technical_terms SET isFavorite = 0")
    suspend fun clearAllFavorites()

    @Query("UPDATE technical_terms SET isFavorite = :favorite WHERE id = :termId")
    suspend fun toggleFavorite(termId: Int, favorite: Boolean)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(terms: List<TechnicalTerm>)

    @Update
    suspend fun updateTerm(term: TechnicalTerm)

    @Query("SELECT COUNT(*) FROM technical_terms")
    suspend fun getCount(): Int

    @Query("SELECT * FROM technical_terms ORDER BY RANDOM() LIMIT 1")
    suspend fun getRandomTerm(): TechnicalTerm?
}
```

## Source Code: TechnicalTerm.kt

```
package com.example.nallanudi.data

import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey

@Entity(
    tableName = "technical_terms",
    indices = [Index(value = ["englishWord"])]
)
data class TechnicalTerm(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val englishWord: String,
    val kannadaWord: String,
    val definition: String,
    val kannadaDefinition: String = "", // New field
    val subject: String,
    val example: String = "",
    val isFavorite: Boolean = false
)
```

## Source Code: TermRepository.kt

```
package com.example.nallanudi.repository

import com.example.nallanudi.database.dao.TechnicalTermDao
import com.example.nallanudi.database.entity.TechnicalTerm
import kotlinx.coroutines.flow.Flow

class TermRepository(private val termDao: TechnicalTermDao) {

    val allTerms: Flow<List<TechnicalTerm>> = termDao.getAllTerms()

    fun getTermsByCategory(category: String): Flow<List<TechnicalTerm>> {
        return termDao.getTermsByCategory(category)
    }

    fun searchTerms(query: String): Flow<List<TechnicalTerm>> {
        return termDao.searchTerms(query)
    }

    suspend fun getTermById(id: Int): TechnicalTerm? {
        return termDao.getTermById(id)
    }

    suspend fun insertTerm(term: TechnicalTerm) {
        termDao.insertTerm(term)
    }

    suspend fun insertAllTerms(terms: List<TechnicalTerm>) {
        termDao.insertAllTerms(terms)
    }

    suspend fun updateTerm(term: TechnicalTerm) {
        termDao.updateTerm(term)
    }

    suspend fun deleteTerm(term: TechnicalTerm) {
        termDao.deleteTerm(term)
    }

    suspend fun getTermCount(): Int {
        return termDao.getTermCount()
    }
}
```

## Source Code: TextToSpeechHelper.kt

```
package com.example.nallanudi.utils

import android.content.Context
import android.speech.tts.TextToSpeech
import android.util.Log
import java.util.Locale

class TextToSpeechHelper(private val context: Context) :
    TextToSpeech.OnInitListener {

    private var textToSpeech: TextToSpeech? = null
    private var isInitialized = false

    init {
        textToSpeech = TextToSpeech(context, this)
    }

    override fun onInit(status: Int) {
        if (status == TextToSpeech.SUCCESS) {
            val result = textToSpeech?.setLanguage(Locale.ENGLISH)
            isInitialized = result != TextToSpeech.LANG_MISSING_DATA &&
                result != TextToSpeech.LANG_NOT_SUPPORTED
            if (isInitialized) {
                Log.d("TTS", "Text-to-Speech initialized successfully")
            }
        } else {
            Log.e("TTS", "Failed to initialize Text-to-Speech")
        }
    }

    fun speak(text: String) {
        if (isInitialized && !text.isBlank()) {
            @Suppress("DEPRECATION")
            textToSpeech?.speak(text, TextToSpeech.QUEUE_FLUSH, null)
        }
    }

    fun stop() {
        textToSpeech?.stop()
    }

    fun shutdown() {
        textToSpeech?.shutdown()
    }

    fun isSpeaking(): Boolean {
        return textToSpeech?.isSpeaking ?: false
    }
}
```

